

# KVB\_LCD: A Frameless GUI Library for STM32F407 and ATmega128

---

## Architecture Defying Limitations

When it comes to developing graphical user interfaces (GUIs) for microcontrollers, embedded engineers usually face a strict compromise: either deploy "heavyweight" libraries like **LVGL**—which demand megabytes of **RAM** for **framebuffers** and a powerful **Cortex-M** core—or settle for primitive graphics suited for basic **8-bit** chips.

The **KVB\_LCD** project proves that with a deep understanding of geometry and display RAM architecture, you can achieve advanced **visual effects in real time using integer-only math** and **exactly 0 bytes of framebuffer RAM**. Graphical objects function absolutely identically on both the **32-bit STM32F407** and the popular **8-bit ATmega128**.

---

## Ultimate Proof: Rendering Speed

Instead of a thousand words, see for yourself how a microcontroller with zero **framebuffer** RAM redraws highly complex scenes in real time:

- **Watch the demo on RUTUBE:**

<https://rutube.ru/video/80140af8b045d20b939240ea3dab1d8b/>

- **Watch the demo on VK Video:**

[https://vkvideo.ru/video-241941150\\_456239018](https://vkvideo.ru/video-241941150_456239018)

- **Watch the demo on YouTube:**

<https://youtu.be/XuP7KXyDUP8>

- **The complete project source code and documentation are available for download here:**

<https://github.com/Konstantin-kvb/KVB-Microcontroller-GUI-Library/>

**Demo Video Details:** The video demonstrates the rendering of **12** complex "three-layer" objects. Each object consists of **3** independent circular/rounded gradient surfaces. This totals **36 independent gradient surfaces** overlaid with **12** gradient characters featuring true

shadow effects. To enhance the lighting and depth depth-of-field perception, the center surface is offset relative to the outer boundary. **All of this is rendered directly into the display's GRAM on the fly, without any framebuffer.**



---

## License

This graphical library is distributed under the **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)** license.

You are free to use, modify, and distribute this code for personal, educational, and non-commercial purposes, provided that proper authorship attribution is given. **Commercial use of this library is strictly prohibited.** To acquire a commercial license, please contact the author directly at [kboruzdov@mail.ru](mailto:kboruzdov@mail.ru) or via phone at +7 965-363-45-29.

---

## Intellectual Property Protection & Proprietary Algorithms

The mathematical approaches and rendering algorithms implemented in the library's core are the unique intellectual property of the author. Specifically, the proprietary algorithm for on-the-fly construction and fill-rendering of a gradient circle within a three-layer object using exclusively integer math (without a framebuffer) is a closed-source know-how that

required immense engineering effort. This is a commercially valuable technology, and the author does not intend to provide these unique mathematical solutions to large corporations free of charge.

To protect copyright and prevent unauthorized reverse engineering of these low-level solutions, the **core graphics engine is delivered strictly as pre-compiled static binary modules** (.a and .lib library files).

The source code of the mathematical core (including high-speed line rendering, GRAM gradient calculation, and three-layer object algorithms) **will not be disclosed under any circumstances**, including the purchase of a commercial license. However, developers get full access to the source code of the high-level GUI wrapper (**KVB\_LCD.h**), which is comprehensively commented in Russian and contains detailed descriptions of all functions, parameters, font effects, and graphical objects. All demonstration code includes extensive comments for a quick and seamless start.

---

## Cross-Platform Compatibility

The library is written from scratch in pure **C** (compatible with **GCC** and **AC6**), utilizing integer-only math without any third-party code dependencies.

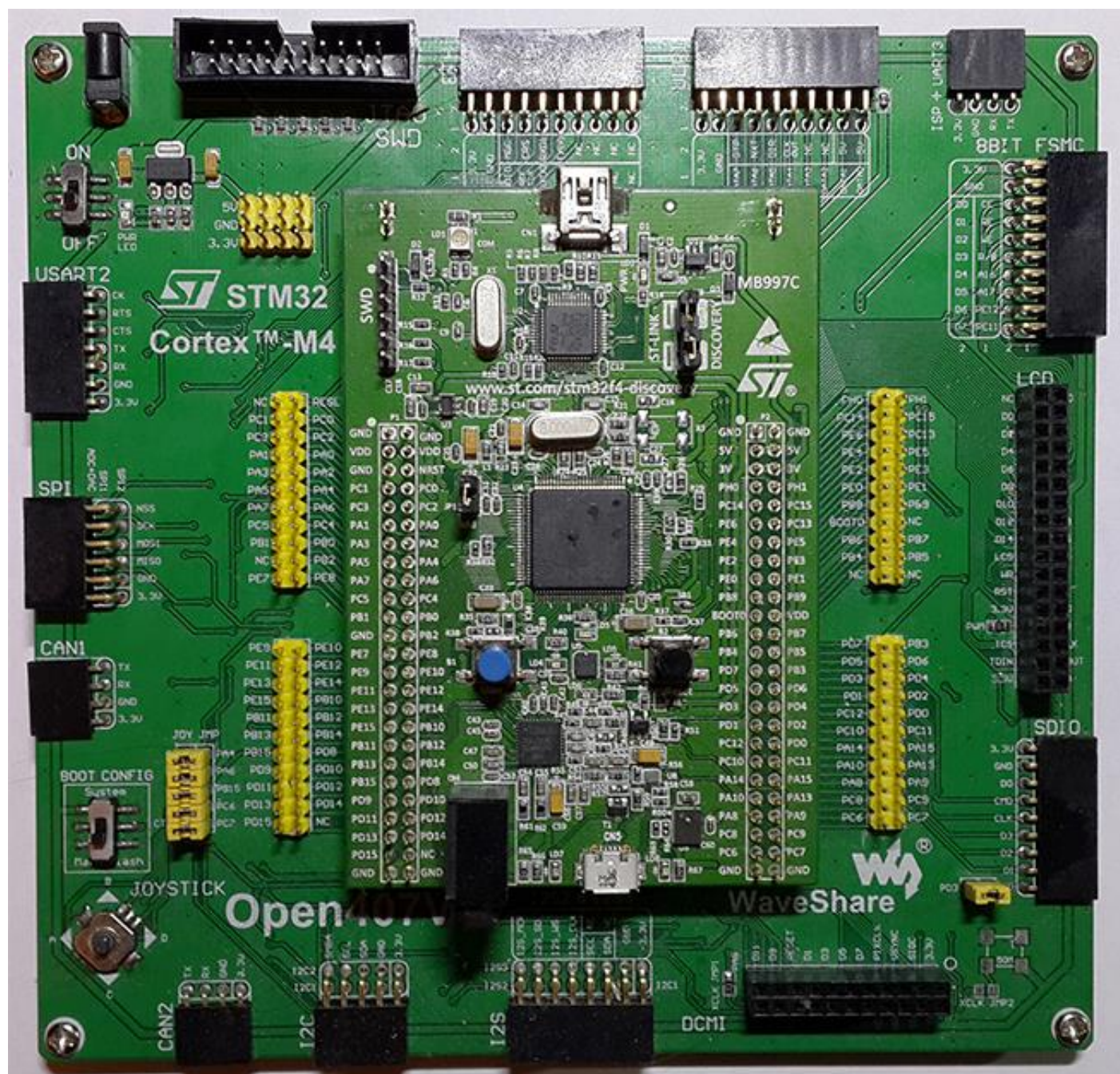
Currently, the library supports **ATmega128** and **STM32F407** microcontrollers at the high-level **GUI** wrapper layer (**KVB\_LCD.h**). Graphical objects built using this library run identically on both **8-bit (ATmega128)** and **32-bit (STM32F407)** MCUs.

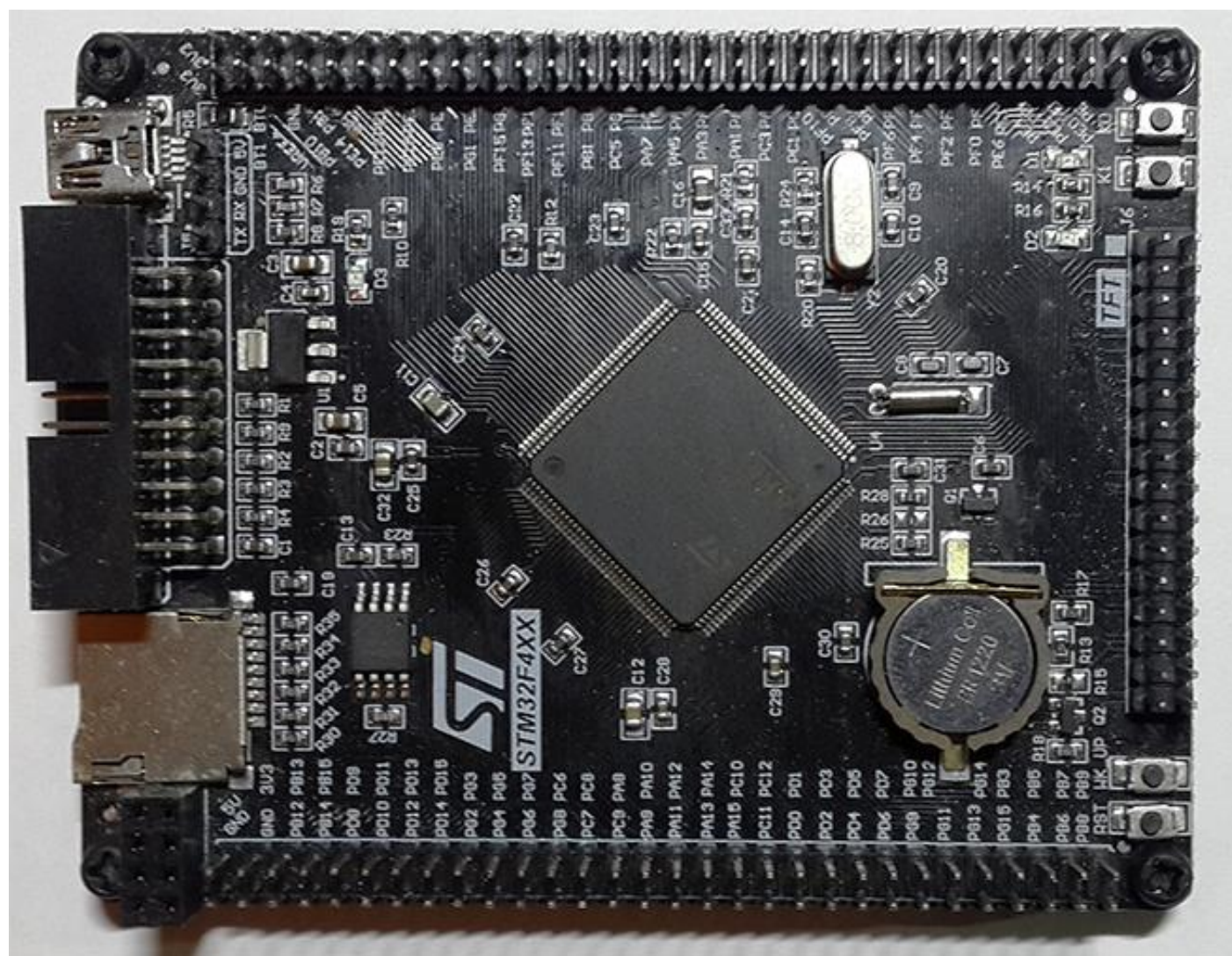
### Supported IDEs:

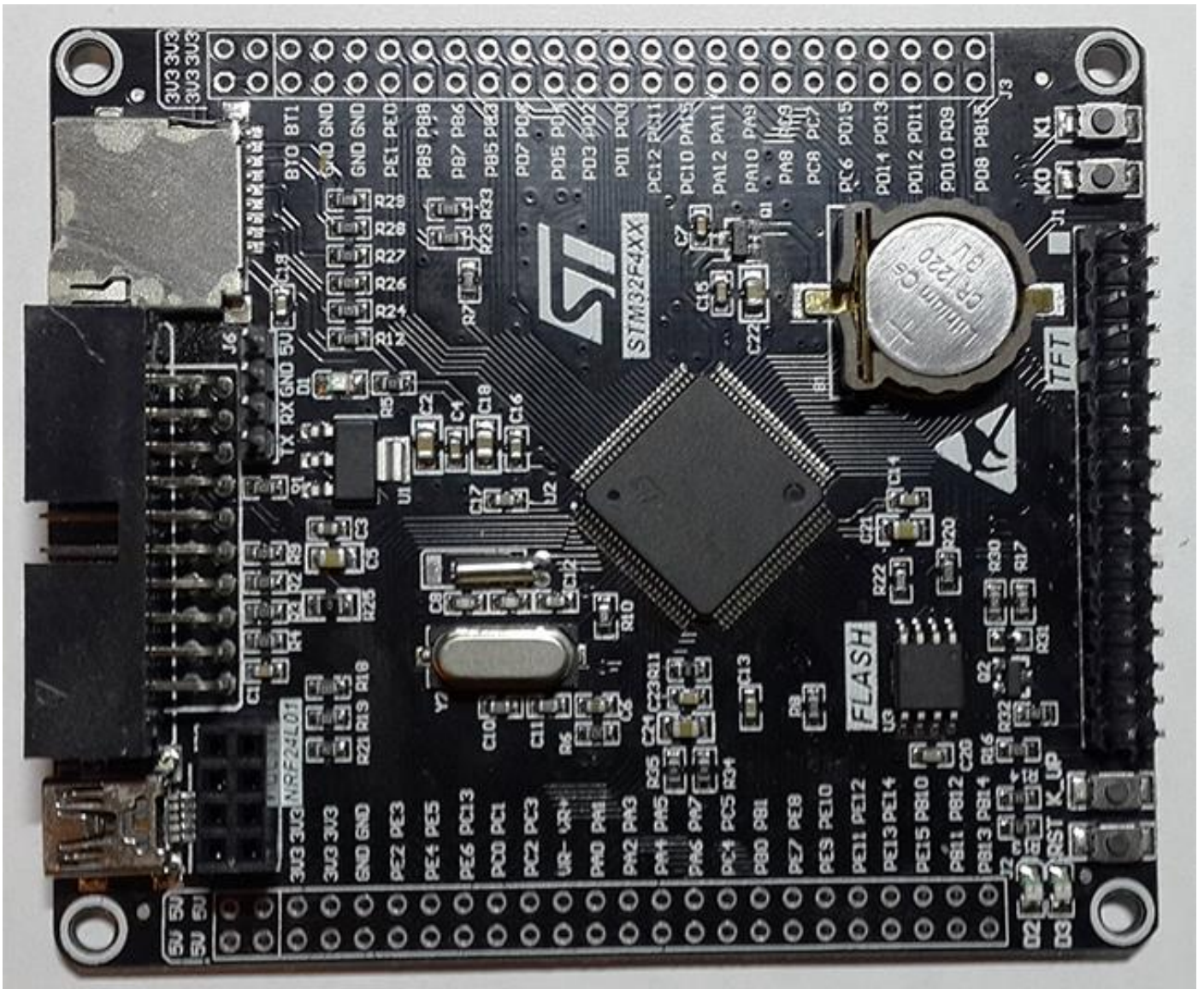
- Atmel Studio 7.0
- STM32CubeIDE
- Keil MDK v5.43

### Supported STM32 Development Boards (with FSMC interface) for evaluation:

- **Open407V-D / stm32f4-discovery** (MCU: STM32F407VGT6)
- **ST STM32F4XX Black v3.0 1606** (MCU: STM32F407ZGT6)
- **ST STM32F4XX Black v2.0 1509** (MCU: STM32F407VET6)

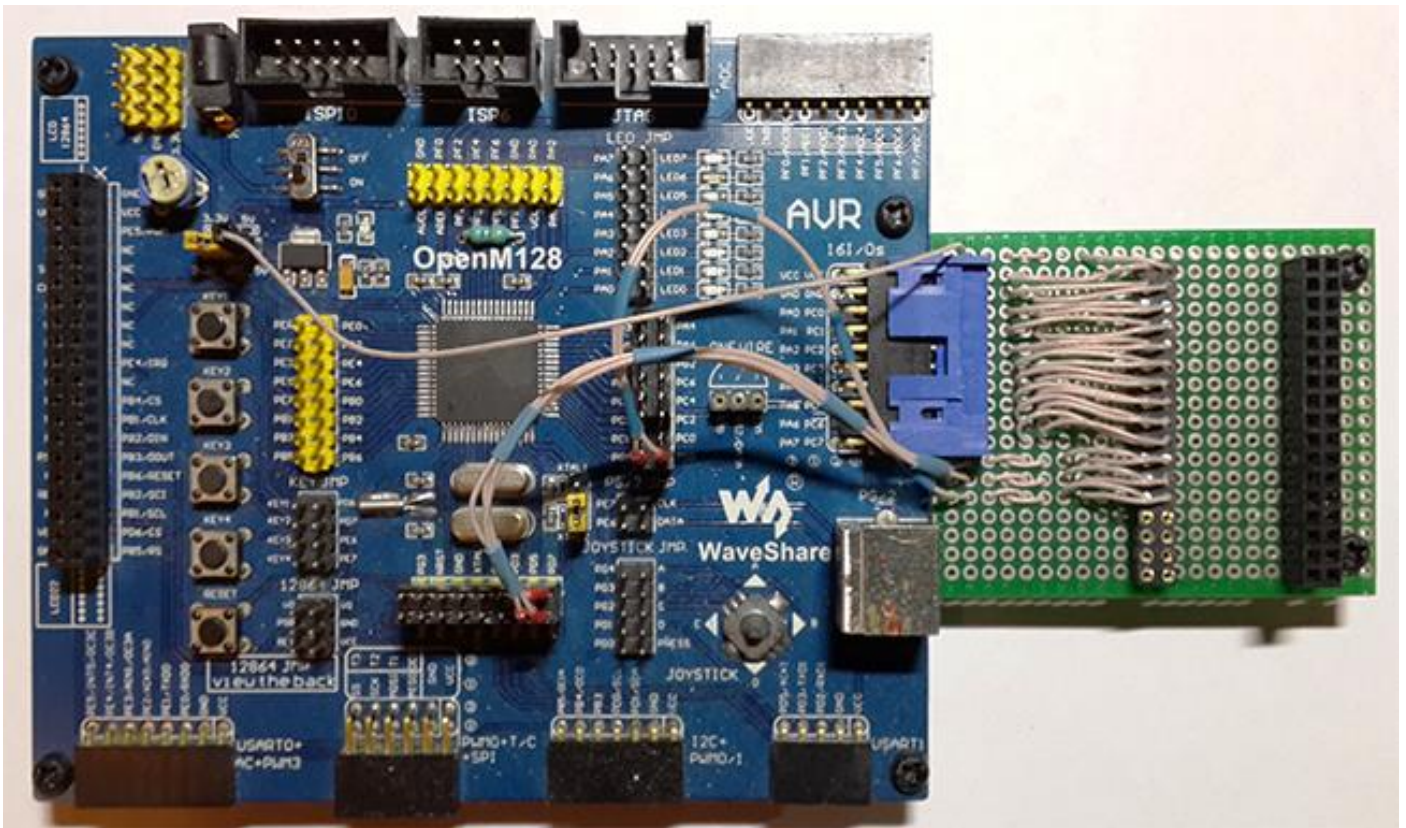






### Pinout Configuration for ATmega128 Evaluation (GPIO Mode):

- **PORTC (D0-D7):** LCD Low Data Byte Bus
- **PORTA (D8-D15):** LCD High Data Byte Bus
- **PORTD bit 4 (CS):** LCD Chip Select Signal
- **PORTD bit 5 (RD):** LCD Read Signal
- **PORTD bit 6 (WR):** LCD Write Signal
- **PORTD bit 7 (RS):** LCD Register Select (Command/Data) Signal
- **PORTG bit 0 (RST):** LCD Reset Signal
- **PORTG bit 1 (BL):** LCD Backlight Control Signal



---

## Supported LCD Panels (Built-in Drivers):

- ILI9325 (320 x 240 resolution)
- ILI9341 (320 x 240 resolution)
- ILI9481 (480 x 320 resolution)
- ILI9484 (480 x 320 resolution)
- OTM8009A (800 x 480 resolution)

*Note: Board and panel pinout mappings are provided as .xlsx spreadsheets in the FSMC LCD directory. While your specific LCD pinout may vary, the board-side mappings remain standard.*

---

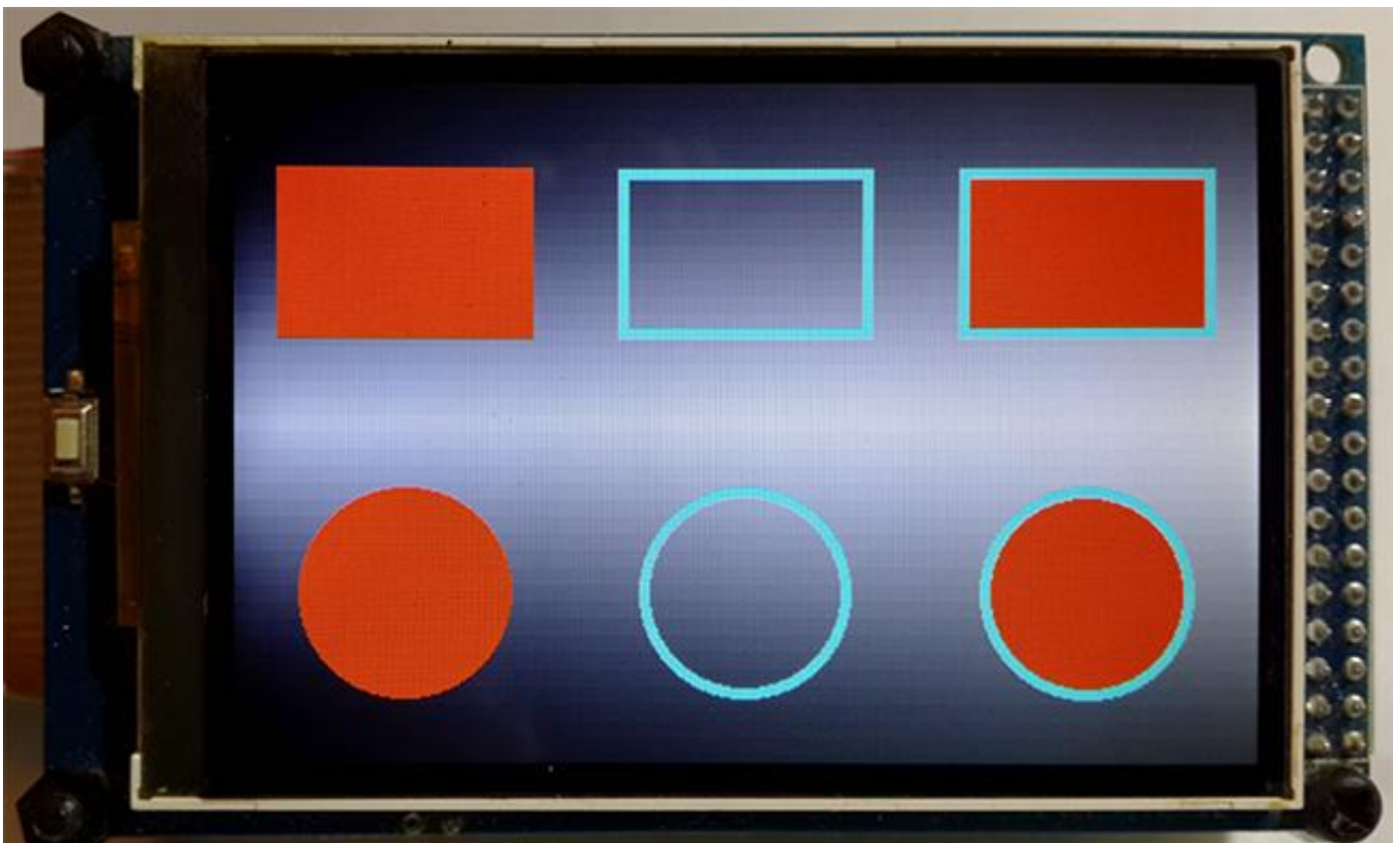
## Unique Features & Proprietary Algorithms

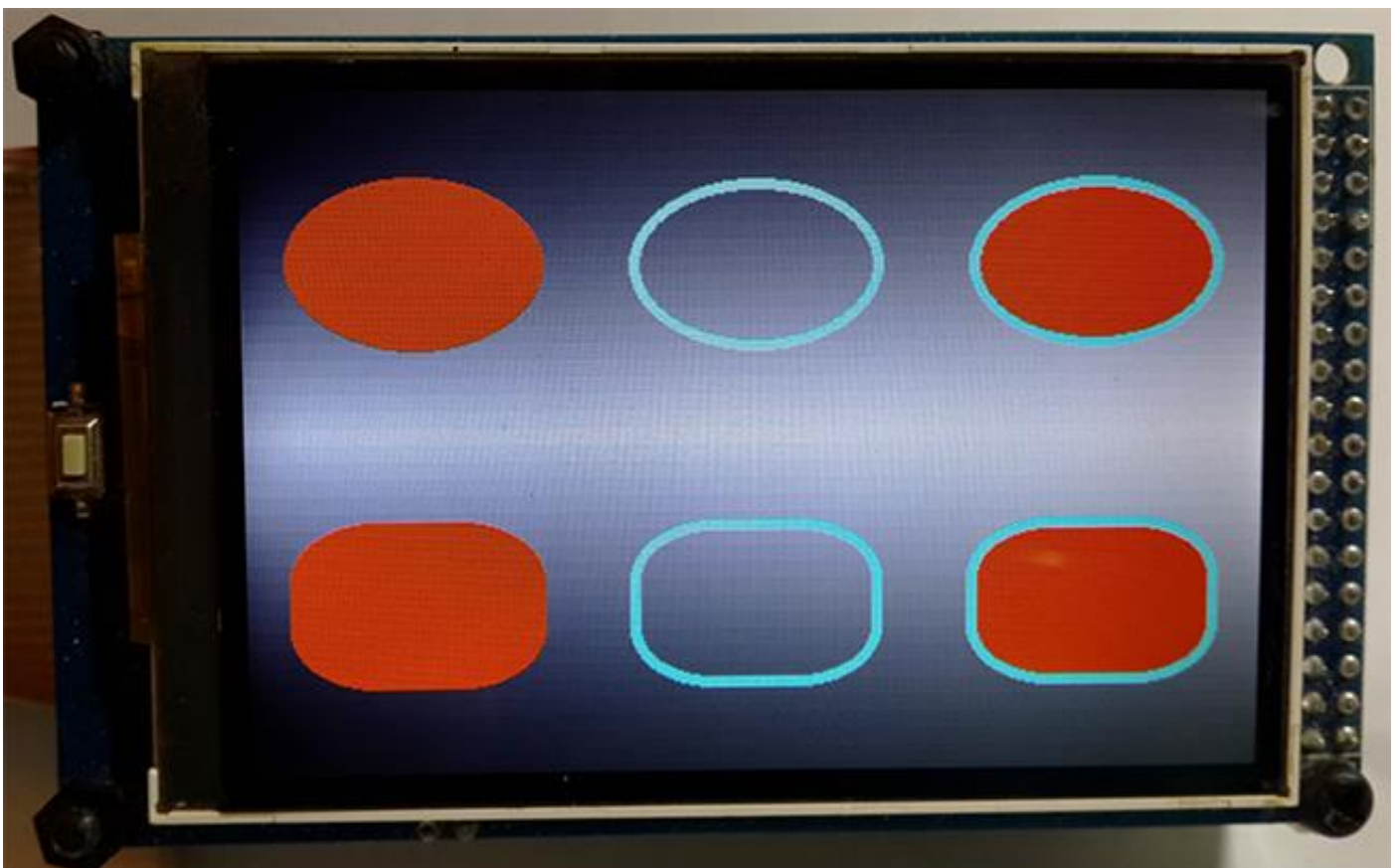
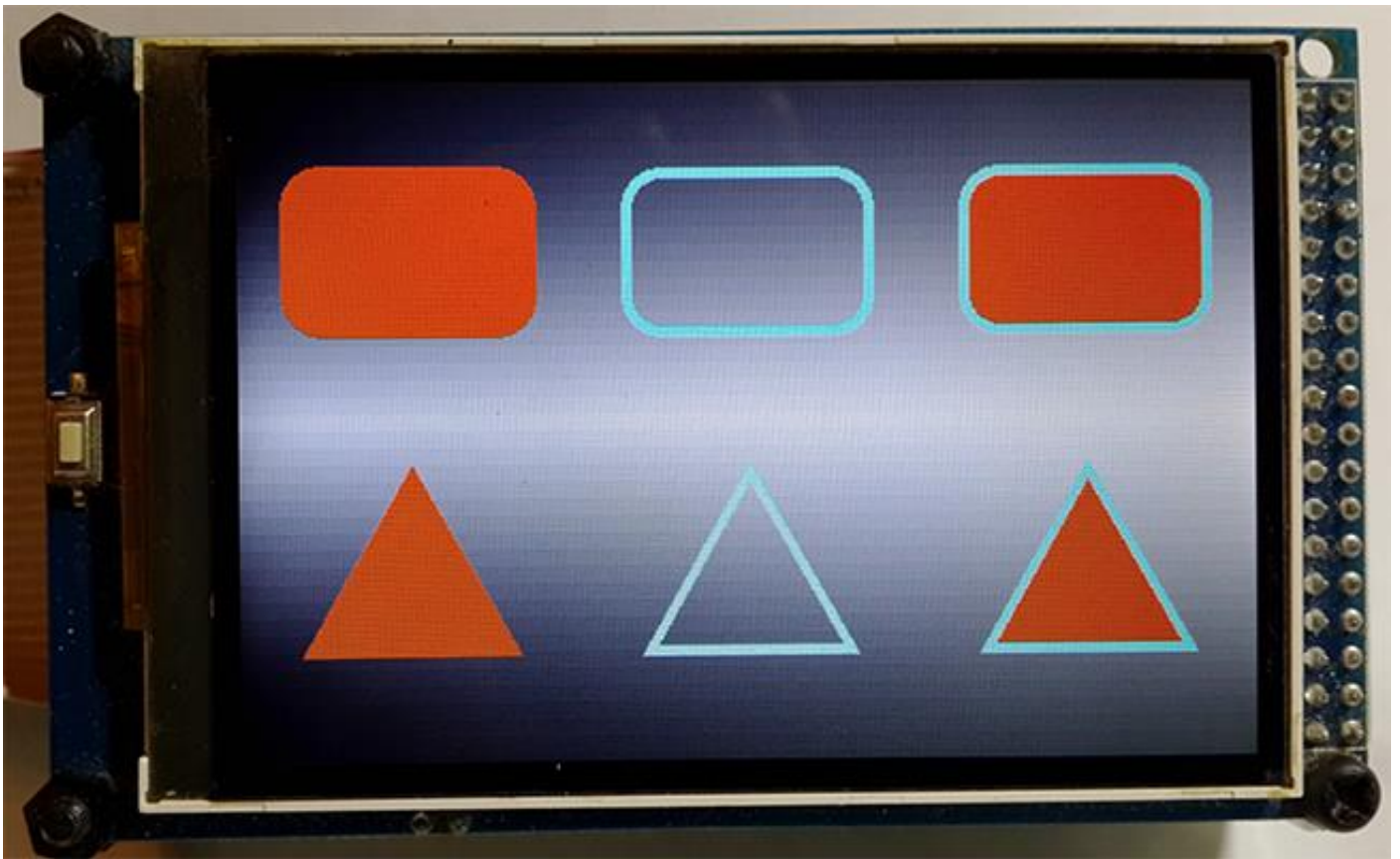


Apart from cross-platform consistency, the primary goal of this library is **maximum rendering speed**. Unlike traditional graphics libraries that rely on standard pixel-by-pixel output, KVB\_LCD features optimized custom algorithms designed around the hardware architecture of display Graphical RAM (GRAM):

- **High-Speed Line Rendering:** A proprietary algorithm calculates contiguous straight segments within inclined/diagonal lines. This minimizes the number of GRAM address switching commands sent to the LCD controller and utilizes fast block fills, exponentially increasing rendering speed.
  - **Gradient Color Fills:** A proprietary algorithm calculates and executes vertical and horizontal gradient fills with linear or centered orientation. It leverages the display hardware matrix to accelerate color rendering inside the GRAM.
  - **Multifunctional Three-Layer Objects:** A proprietary algorithm handles integer-based circle generation with on-the-fly gradient fills without a framebuffer. This object renders three nested rounded, circular, or rectangular surfaces simultaneously, each with gradients in different directions. All three surfaces can be redrawn independently. For example, you can visually highlight an object selection by redrawing only the outer border color, skipping the inner surfaces completely to save processing time.
- 

## Advanced Graphical Primitives





Even basic geometric figures in KVB\_LCD offer more advanced features than standard alternatives. They include:

- Adjustable border width.
  - A configurable center surface.
  - **Three rendering modes:** Border-only, Fill-only, or Full Object. This allows object selection/interaction handling by changing just the border color on the fly, preventing costly redraws of the inner area.
  - **Four-way vertex orientation** for triangle primitives (both equilateral and isosceles): Left, Right, Up, or Down.
- 

## Advanced Text Rendering Subsystem



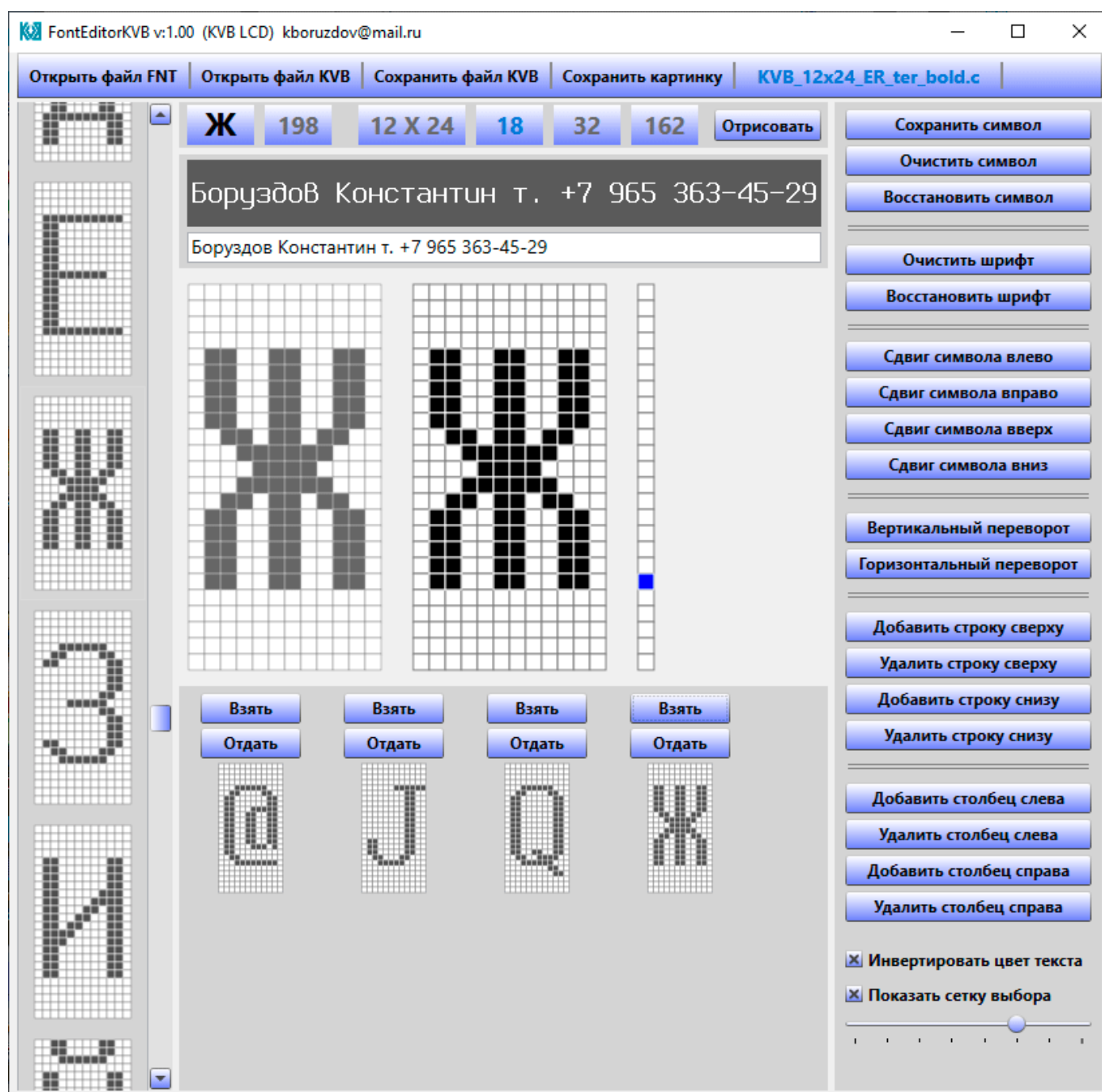


The engine features a powerful, high-performance text rendering subsystem that automatically handles line alignment and applies complex visual effects on the fly without a framebuffer:

- **Baseline Anchoring:** All fonts align to a single horizontal guide line (the character baseline). This allows words of different sizes and colors to be combined into a single text line while maintaining perfect vertical alignment.
- **Automatic Word Wrapping:** Long continuous text blocks—which can consist of multiple string, integer, and float variables of varying font sizes and colors—automatically wrap to the next line when exceeding text block boundaries.
- **Tabulation Support:** Native handling of `\t` tab characters.
- **Configurable Font Kerning:** Custom character spacing with automatic length correction for spaces and tabs.
- **Adjustable Line Spacing:** Supports both positive and negative spacing offsets. Negative values optimize layouts for ALL-CAPS text blocks, while positive values improve general text readability.
- **Font Scaling Factor:** Dynamic integer scaling of font assets.
- **Text Alignment:** Align text to Left, Right, or Center.

- **Shadow Effects:** Four configurable shadow orientations (Bottom-Right, Bottom-Left, Top-Right, Top-Left). This creates professional embossed or debossed font effects, significantly increasing text readability.
- **Character Outlining:** Renders a crisp circular outline around each glyph to ensure text remains legible on any background pattern. Can be set to render the outline only, omitting the character fill.
- **Gradient Text:** Renders smooth horizontal or vertical color transitions using linear or centered text line fills, fully compatible with shadow and outline effects.

## Font Editor (Windows Desktop Tool)



To streamline the creation and customization of display fonts, a dedicated Windows desktop application was developed to automate the asset preparation workflow:

- **DOS Font Import (.fnt):** Allows rapid importing of classic bitmapped .fnt files from the DOS era for subsequent optimization and adaptation to microcontroller screens.
- **Pixel-Grid Editing:** Provides a convenient grid-based GUI for manual pixel drawing, glyph correction, and baseline adjustment.
- **Interactive Real-Time Rendering:** Renders a sample text string dynamically. You can see exactly how altering a single pixel in a glyph changes the look of the entire text string in real time.
- **Export to C Source Code:** Generates optimized .c source files with ultra-compact sequential data packing. A standard 9x16 glyph takes up **just 18 bytes**, ensuring full compatibility with AVR's PROGMEM Flash storage or standard const arrays in STM32.
- **Proprietary Font Pack:** The author utilized this editor to craft **23 unique dual-language (Russian/English) fonts**. (5 base fonts are bundled with the demo presentation; the complete font collection and full editor are available upon request).

*Note: The evaluation version of the Font Editor has the "Export to C-format" feature disabled. The fully unlocked version is available upon request.*

---

Best regards,  
Konstantin Boruzdov

**Contact Information:**

-  **Email:** kboruzdov@mail.ru
  -  **Tel:** +7 (965) 363-45-29
-